MalariaScope

Architecture

Kiran Kumar P
DA24B008

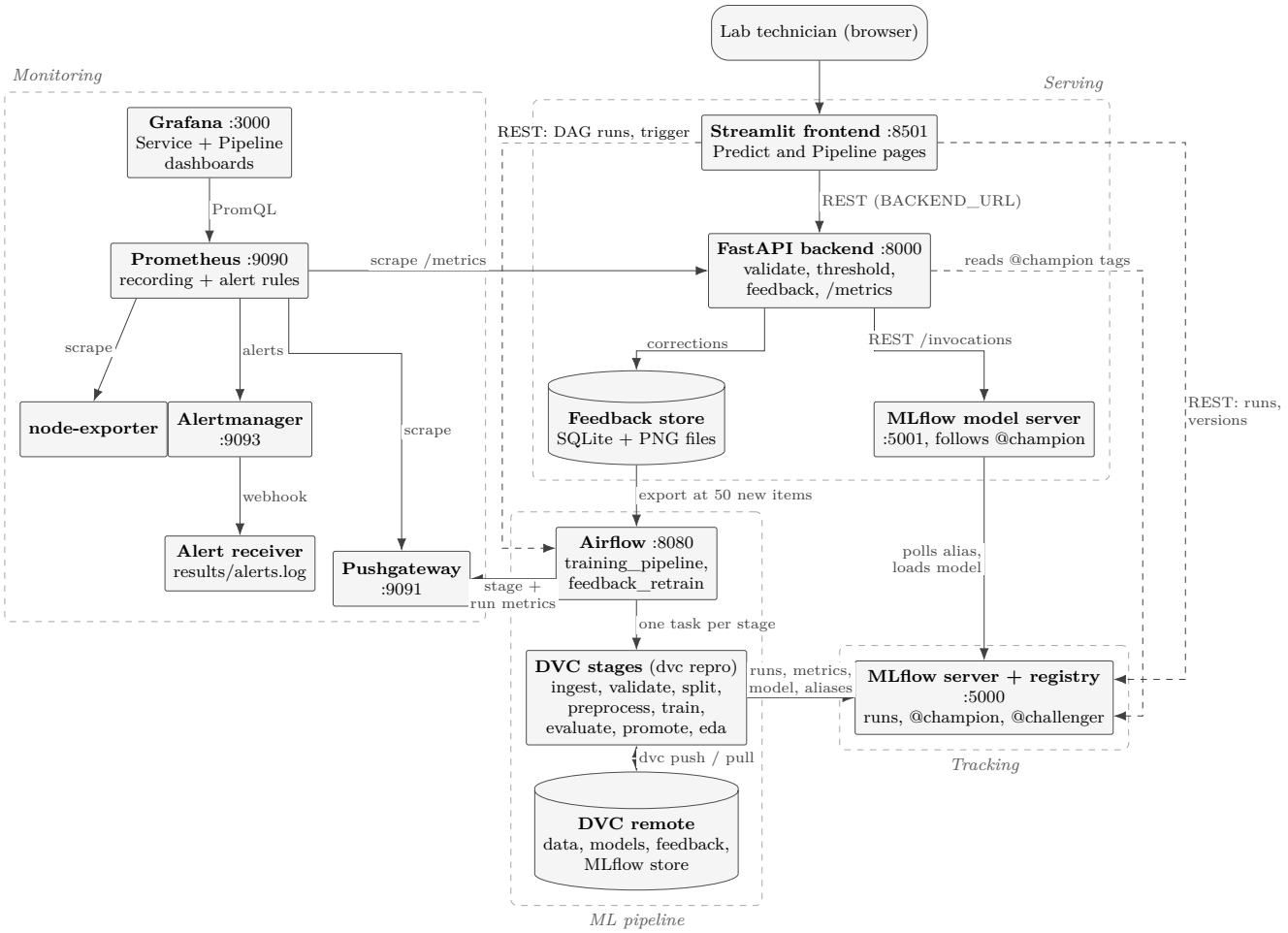


Figure 1: Blocks and the connections between them. Solid arrows are the data path, dashed arrows are read-only lookups and pipeline controls from the UI.

Each block is its own container under Docker Compose (12 services, all with healthchecks). They talk only over REST or through files they own, and every address comes from an environment variable. Host ports are bound to 127.0.0.1.

Serving

- **Streamlit frontend (:8501):** upload a cell image or pick a sample, see the label and confidence, and send a correction. A second page shows the pipeline: Airflow runs, the model in use, recent training runs, feedback counts and links to every tool's UI. The UI knows the backend only through `BACKEND_URL`.
- **FastAPI backend (:8000):** checks the upload (PNG or JPEG, at most 5 MB), resizes it with the same `to_model_input()` function the pipeline uses, sends the pixels to the model server, applies the champion's tuned threshold (0.2417 for v1) and returns label, confidence and model version. It also stores feedback and exposes `/metrics`.
- **MLflow model server (:5001):** runs `mlflow models serve` for the version behind `@champion`. A small wrapper checks the alias every 15 s and restarts the server when it moves, so a promotion or a rollback needs no manual restart.

- **Feedback store:** SQLite rows plus one PNG per item under `images/<label>/`. Items not yet used for training are counted as pending.

Tracking

- **MLflow server and registry (:5000):** SQLite backend with proxied artifacts. It holds every run (params, per-epoch metrics, plots, the model with its signature) and the registered versions of `malariascope` with the aliases `@champion`, `@challenger` and `@previous_champion`. Each run is tagged with the git commit and the DVC hash of its data.

ML pipeline

- **DVC stages:** ingest → validate → split → preprocess → train → evaluate → promote, with eda next to them. Params live in `params.yaml` and every input and output is hashed in `dvc.lock`, so a stage only reruns when something it depends on changed. Validate and preprocess use Ray Data (about 5,400 and 5,100 images/s on 16 cores).
- **Airflow (:8080):** `training_pipeline` has one `dvc repro <stage>` task per stage and ends with `dvc push`. `feedback_retrain` runs every 30 minutes and starts a retrain once 50 new feedback items are waiting.
- **DVC remote:** a local folder that holds the raw and processed data, the model, the feedback snapshots and a copy of the MLflow store.

Monitoring

- **Prometheus (:9090):** scrapes the backend, node-exporter and the Pushgateway every 5 s, and evaluates 9 recording rules and 10 alert rules.
- **Pushgateway (:9091):** pipeline tasks finish before a scrape would reach them, so each Airflow task pushes its stage metrics here, and each DAG run pushes its result.
- **Alertmanager (:9093) and alert receiver:** firing alerts go through Alertmanager to a small webhook that writes one line per alert to `results/alerts.log`.
- **Grafana (:3000):** two provisioned dashboards. *Service* shows traffic, latency, predictions, feedback accuracy and input drift. *Pipeline* shows DAG runs, the promotion decision, training results and Ray throughput.

Request path

Browser → Streamlit → `POST /predict` on the backend → `POST /invocations` on the model server → p(parasitized) back to the backend, threshold applied → label and confidence on screen. Through the containers this takes 65.6 ms at the median and 196.0 ms at p95 (50 calls, idle laptop).

Model path

TFDS `malaria` (27,558 images) → DVC stages run by Airflow → new MLflow run and registered version (`@challenger`) → promotion gate against `@champion` on the same test split → if it wins, the alias moves and the model server switches within about 20 seconds. Feedback re-enters at the split stage, into the train split only.